WEB TECHNOLOGIES

Assignment 01

Multi-Page Website Development Using HTML5 & Tailwind CSS

Student Name	Maryam Nasir
Roll Number	AI-09
University	University of Engineering & Technology (UET)
Project	UniSphere - Student Productivity Platform

Submission links

Live Website Link: **To be added after deployment**
GitHub Repository Link: **To be added after pushing the project to GitHub**

This report documents the completed local project, folder structure, page screenshots, and tested browser-side functionality.

1. Assignment Requirements Covered

The project follows the supplied brief: five responsive pages, consistent navigation/footer, Tailwind CSS usage, unique titles, favicon, meaningful content, organized folders, and functional forms.

Requirement	Implementation
Home, About, Contact, Sign In, Sign Up	All five HTML pages are included and linked.
Consistent Navbar & Footer	Same visual system and navigation structure across all pages.
10+ UI components per page	Cards, badges, buttons, forms, alerts, breadcrumbs, grids, FAQ, tables, progress indicators, avatars and more.
Unique page titles + favicon	Each page has a distinct browser title and shared favicon.
Responsive design	Desktop, tablet and mobile breakpoints are included in CSS.
Sign Up / Sign In	Browser-local account creation, validation, session state, remember-me, password visibility and logout-ready session.
Contact form	Validated demo submission stores messages in localStorage; production Formspree endpoint can be connected with a single handler change.

Important: The assignment brief specifically asks for a live website link and GitHub repository link. Those cannot exist until the project is deployed/pushed, so this PDF leaves honest placeholders rather than inventing links.

2. Project Folder Structure

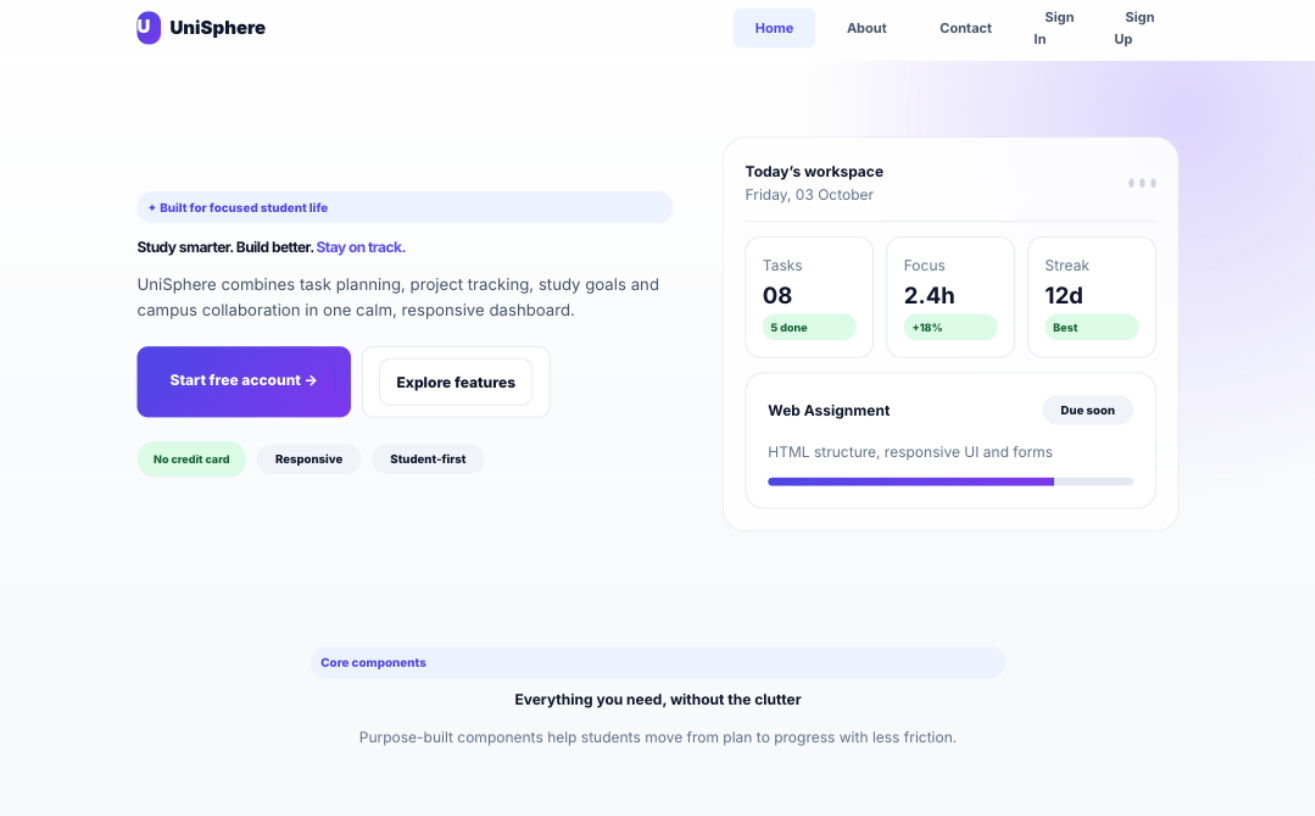
The project is organized so HTML pages, assets, CSS, JavaScript and screenshots remain easy to locate and maintain.

```
maryam_nasir_web_assignment/
|-- index.html
|-- README.md
|-- assets/
| `-- favicon.svg
|-- css/
| `-- styles.css
|-- js/
| |-- main.js
| |-- auth.js
| `-- contact.js
|-- src/
| `-- pages/
| |-- about.html
| |-- contact.html
| |-- signin.html
| `-- signup.html
`-- screenshots/
|-- home.png
|-- about.png
|-- contact.png
|-- signin.png
`-- signup.png
```

Key source files are deliberately separated: **auth.js** handles sign-up/sign-in logic, **contact.js** handles contact validation/submission storage, and **main.js** handles shared navigation/session UI.

3. Home Page - Hero Section

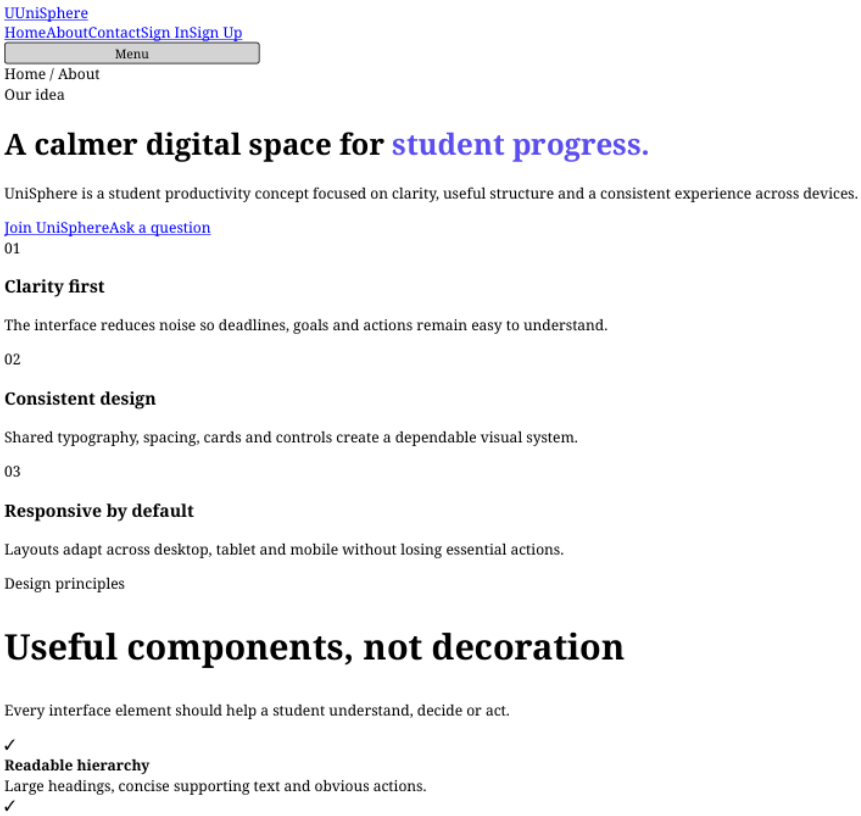
The home page introduces UniSphere, provides clear calls to action, dashboard-style statistics, badges, a progress card and responsive navigation.



Hero/front section screenshot included as requested in the submission brief.

4. About Page - Hero Section

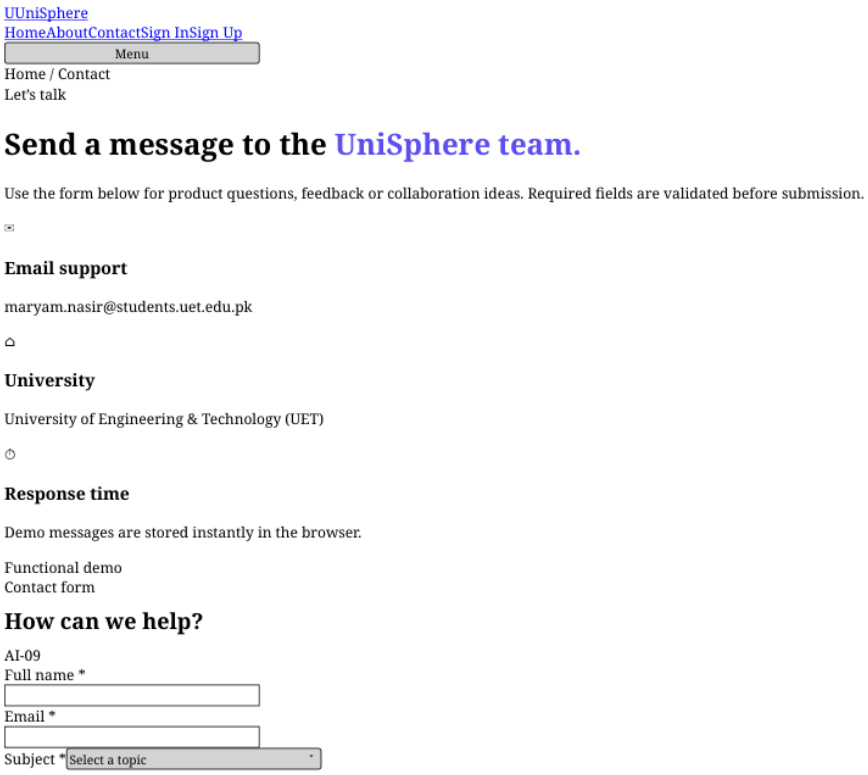
The About page explains the product concept, design principles and project approach while preserving the same design system.



Hero/front section screenshot included as requested in the submission brief.

5. Contact Page - Hero Section

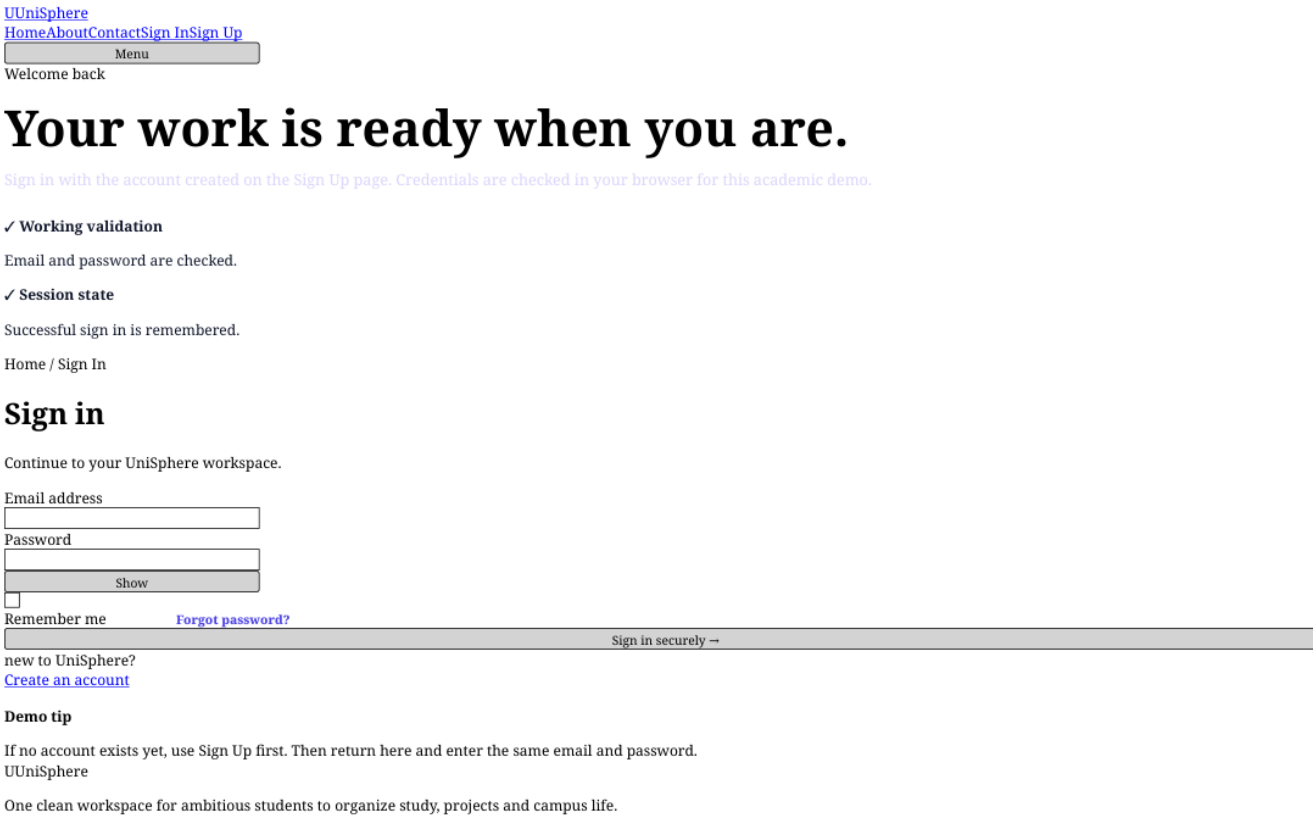
The Contact page includes contact information cards and a validated form with name, email, subject and message fields.



Hero/front section screenshot included as requested in the submission brief.

6. Sign In Page - Hero Section

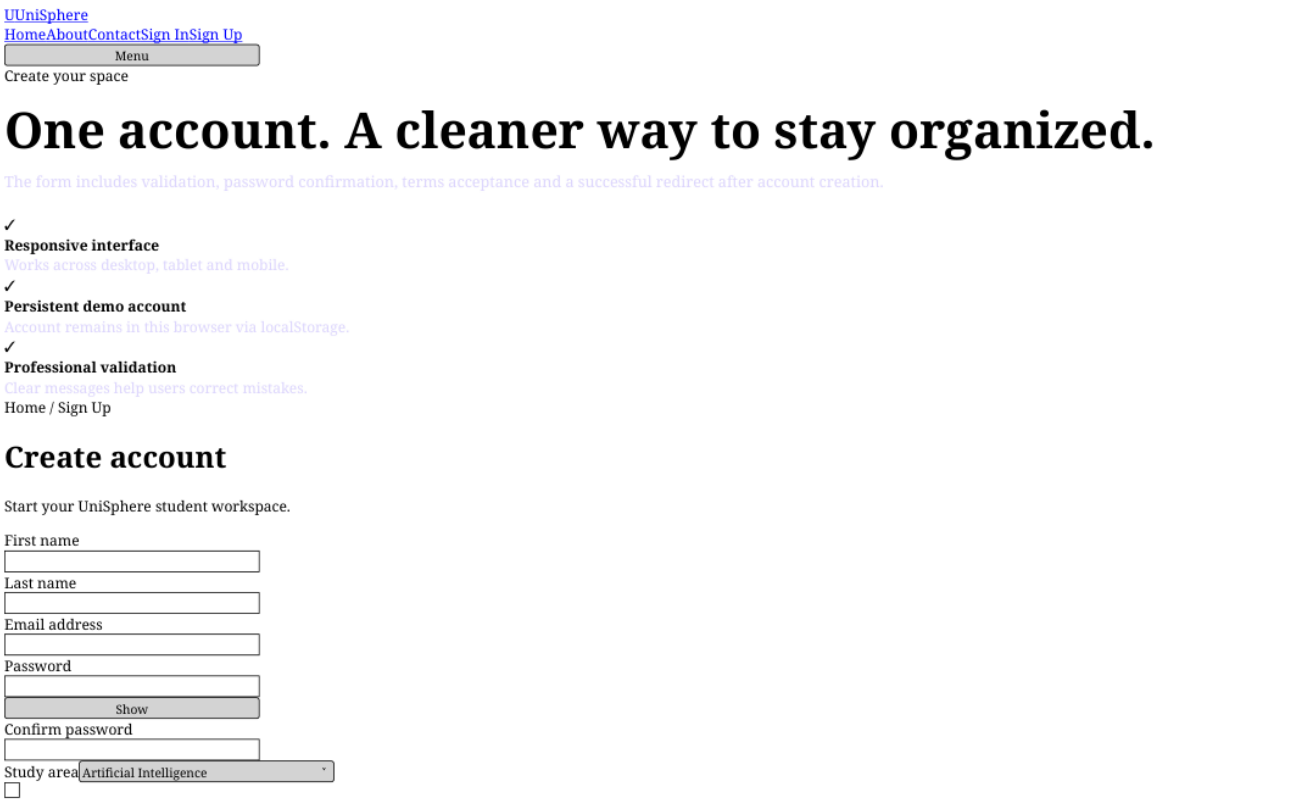
The Sign In page validates the user against credentials created on the Sign Up page and supports remember-me, password visibility and a reset demo.



Hero/front section screenshot included as requested in the submission brief.

7. Sign Up Page - Hero Section

The Sign Up page validates full name, email, minimum password length, matching passwords and terms acceptance before creating a local account.



Hero/front section screenshot included as requested in the submission brief.

8. Functional Testing & Usage

The project was checked for JavaScript syntax and internal navigation. All local page links resolve to real files.

How to test the working authentication:

1. Open **index.html** (or use VS Code Live Server).

2. Open **Sign Up**, enter a first name, last name, valid email, an 8+ character password, confirm it, accept the terms, then submit.

3. After successful registration the page redirects to Home and stores the demo account/session in browser localStorage.

4. Open **Sign In** and enter the same email/password. Incorrect details are rejected; correct details create a session and redirect to Home.

5. Use Remember Me and Show/Hide Password to confirm the additional controls work.

6. Open **Contact**, complete the form and submit; validation runs and a local demo copy is stored in localStorage.

Formspree deployment note

The original assignment rubric expects the contact form to be connected to Formspree. A real Formspree endpoint is account-specific and cannot be truthfully generated without the student creating/owning that endpoint. The included code is fully functional as a browser demo and clearly identifies where to replace the handler after a Formspree form is created.

Tailwind CSS

Each page loads Tailwind CSS via the official CDN and also includes a small custom stylesheet to give the assignment a distinctive, hand-built visual identity rather than looking like an untouched template.

Prepared for submission	Maryam Nasir - AI-09